

Лекция 2. Инструменты: как дать агенту руки

22 сентября 2026

О чём эта лекция

- Почему модель зовёт функции, которых нет, и почему одна задача через разные API даёт разные вызовы
- Что схема аргументов гарантирует — и что она не гарантирует ничего про смысл
- Как выбрать между двумя описаниями инструмента, не споря: прогнать тридцать задач и сравнить доли верных вызовов
- Что фреймворк делает за вас: цикл, историю, диспетчер вызовов, передачу между агентами
- Какие решения он за вас не примет: описания, формат результата, что считать нарушением
- Кто разговаривает с пользователем, когда агентов несколько

Где мы остановились: цикл стал одной строкой

ПЕРВАЯ ЛЕКЦИЯ: ЦИКЛ РУКАМИ

```
messages = [{"role": "user", "content": task}]
for step in range(max_steps):
    r = client.chat.completions.create(
        model=MODEL, messages=messages, tools=tools)
    msg = r.choices[0].message
    if not msg.tool_calls:
        return msg.content
    messages.append(msg)
    for call in msg.tool_calls:
        obs = dispatch(call)
        messages.append({"role": "tool",
                        "tool_call_id": call.id,
                        "content": obs})
```

СЕГОДНЯ: ТОТ ЖЕ ЦИКЛ, НО ЧУЖОЙ

```
agent = Agent(
    name="Городской аналитик",
    instructions="Отвечай числами из базы",
    tools=[find_places, count_places])

result = await Runner.run(
    agent, task, max_turns=6)
print(result.final_output)
```

- Исчезли: история, диспетчер вызовов, лимит шагов, журнал
- Остались: описания инструментов, формат результата, что считать нарушением, что делать с пропусками в данных
- Лекция — про второй список

Что модель генерирует на самом деле

API показывает нам аккуратную структуру `tool_calls`. Модель же выдаёт **текст в служебном формате**:

```
<|assistant|> Нужна статистика по району. <tool_call> {"name": "district_stats", "arguments": {"city":  
"Ростов-на-Дону", "district": "Ленинский"}} </tool_call> <|end|>
```

 маркеры чат-шаблона  служебные токены блока вызова  имя и аргументы

- `tool_calls` в ответе — это **разбор текста**, который сделал сервер провайдера
- Шаблоны у семейств моделей разные: та же задача через другой API даёт другой **ВЫЗОВ**
- Пока блок не дописан до конца, вызова нет — модель просто генерирует токены

Как этому научили

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Schick et al. Toolformer, 2023, рис. 1: модель сама решает вставить вызов калькулятора, поиска, перевода

- **Toolformer (2023):** модель сама вставляла вызовы в текст и оставляла те, что улучшали предсказание
- **Сегодня:** дообучение на траекториях вызовов плюс RL с проверкой результата
- Отсюда всё остальное: модель воспроизводит **формат**, а не понимает инструмент. Имя, описание и схема — единственное, что она о нём знает
- Поэтому инструмент — контракт из двух половин: код ваш, а выбор инструмента и аргументов живёт в модели. Другая модель зовёт иначе, хотя код не тронут

Схема — это контракт

```
{
  "name": "search_listings",
  "description": "Ищет объявления о продаже квартир
    в базе курса. Используй для отбора по
    параметрам; для статистики по району
    используй district_stats.",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string"},
      "district": {"type": "string",
        "description": "район как в базе"},
      "rooms": {"type": "integer", "minimum": 1},
      "max_price_rub": {"type": "integer"},
      "sort": {"type": "string",
        "enum": ["price", "price_per_m2"]}
    },
    "required": ["city"]
  }
}
```

- Типы, `enum`, `required` — что модель обязана выдать
- Описания полей читает модель, а не только программист
- Единицы — прямо в имени: `max_price_rub`, а не `price`
- Схема попадает в контекст **на каждом шаге**: её размер — это тоже стоимость

Валидность по построению: constrained decoding

Схема превращается в автомат. На каждом шаге модель выбирает **только из допустимых токенов**:

ПОЗИЦИЯ	ДОПУСТИМО	ВЫБРАНО
начало	{	{
ключ	"city" "district" "rooms" ...	"district"
после ключа	:	:
значение district	любая строка	"Ленинский"
значение rooms	целое ≥ 1	2
значение sort	"price" "price_per_m2"	"price"

- Гарантия **синтаксическая**: JSON всегда разберётся, типы совпадут, `enum` не нарушится
- Гарантии смысла **нет**: район «Ленинский» валиден, а в задаче был Кировский
- Смысл проверяет код инструмента и шаг проверки в цикле
- Так работают structured outputs у провайдеров и библиотеки вроде XGrammar

Откуда фреймворк берёт схему

ФУНКЦИЯ С АННОТАЦИЯМИ И ДОКСТРОКОЙ

```
@function_tool
def count_places(
    kind: str | None = None,
    open_24_7: bool = False) -> str:
    '''Сколько мест подходит под условие:
    число, а не список.

    Args:
        kind: вид места по-английски:
            pharmacy, cafe, outpost
        open_24_7: только круглосуточные
    '''
```

ЧТО УЕХАЛО МОДЕЛИ

```
{
  "description": "Сколько мест подходит
  под условие: число, а не список.",
  "properties": {
    "kind": {
      "anyOf": [{"type": "string"},
        {"type": "null"}],
      "description": "вид места",
      "open_24_7": {"type": "boolean",
        "default": false,
        "description": "круглосуточные"}},
    "required": ["kind", "open_24_7"]}
}
```

- Первая строка докстроки → описание инструмента, секция `Args:` → описания аргументов
- В `required` **оба поля**, хотя у второго есть значение по умолчанию
- Без докстроки модель не знает об инструменте ничего, и никто не предупредит

Описание — это интерфейс

ТАК ПИШУТ ОБЫЧНО

```
{"name": "search",  
  "description": "Поиск объявлений"}
```

```
{"name": "stats",  
  "description": "Статистика"}
```

```
{"name": "predict",  
  "description": "Предсказание цены"}
```

ТАК МОДЕЛЬ НЕ ПРОМАХНЁТСЯ

```
{"name": "search_listings",  
  "description": "Отбор объявлений о ПРОДАЖЕ по  
    параметрам. Возвращает до 20 строк: id,  
    район, комнаты, площадь, цена. НЕ считает  
    статистику — для медиан по району вызывай  
    district_stats. Цены в рублях."}
```

- Что делает · когда использовать · **когда не использовать** · единицы и формат · что вернёт
- Похожие инструменты различайте явно, в тексте описания
- Имена — с общим префиксом домена: `listings_search`, `listings_stats`
- Спор о формулировке решается замером: те же тридцать задач, два описания, разные доли верных вызовов

Результат инструмента — тоже интерфейс

СЫРОЙ ОТВЕТ БАЗЫ

```
[{"id": 8812, "city": "Ростов-на-Дону",  
  "district": "Кировский", "address": "...",  
  "lat": 47.22, "lon": 39.71, "rooms": 2,  
  "area_total": 54.1, "area_kitchen": 9.0,  
  "floor": 5, "floors_total": 9,  
  "year_built": 1978, "renovation": "косметический",  
  "price": 6400000, "price_per_m2": 118300,  
  "published_at": "2026-08-30", "days_on_market": 14,  
  "seller_type": "owner",  
  "description": "Продаётся уютная... (900 знаков)"},  
... ещё 141 объект]
```

КОМПАКТНАЯ ТАБЛИЦА

142 объявления, показаны 10 по цене за м²:

id	район	комн	м²	₽/м²
8812	Кировский	2	54.1	118 300
8827	Кировский	2	48.0	122 900
...				

Остальные: `search_listings(page=2)`

- Формат — таблица с разделителем (CSV, TSV), а не JSON: те же десять мест — 660 знаков против 1150
- Результат живёт в контексте **всех следующих шагов**: только нужное, пагинация, единицы в заголовках
- Под LLM есть и свой формат — **TOON**: длина и поля объявляются один раз; на плоских таблицах уступает CSV, выигрывает на вложенных

Ошибка — это тоже результат

ИСКЛЮЧЕНИЕ

```
Traceback (most recent call last):  
  File "tools.py", line 41, in search_listings  
    rows = db.execute(q).fetchall()  
sqlite3.OperationalError: database is locked
```

Цикл падает или модель видит трассировку без подсказки

НАБЛЮДЕНИЕ С ПОДСКАЗКОЙ

Ошибка: база временно занята.
Повтори вызов через несколько секунд
или уменьши выборку (укажи район).

- Исключение внутри инструмента → **текст для модели**, цикл продолжается
- Подсказка «что делать дальше» — часть интерфейса
- Пустой результат тоже объясняйте: «0 объявлений: возможно, район назван иначе»

Два способа позвать другого агента

HANDOFF: РАЗГОВОР УХОДИТ

```
triage = Agent(  
    name="Триаж",  
    instructions="Справочные вопросы – "  
        "в справочную, задачи по выбору "  
        "места – аналитику.",  
    handoffs=[reference, analyst])
```

- Второй агент получает разговор и отвечает сам; первый в него больше не вернётся
- Подходит, когда ветки независимы: справка или анализ

АГЕНТ КАК ИНСТРУМЕНТ: РАЗГОВОР ОСТАЁТСЯ

```
analyst = Agent(  
    name="Аналитик",  
    tools=[reference.as_tool(  
        tool_name="справочная",  
        tool_description="факты о городе")])
```

- Второй агент — обычный инструмент: позвали, получили текст, пошли дальше
- Владелец разговора не меняется: ответ собирает первый
- Подходит, когда нужен фрагмент работы, а не передача задачи

Кто пишет финальный ответ. При handoff — второй агент, поправить его уже нельзя. При «агенте как инструменте» — первый: сведёт несколько источников к одному формату, но заплатит за лишнюю генерацию

Оркестрация: решает модель или решаете вы

Решает модель

- Один агент, много инструментов, свобода выбора шагов
- Незнакомая задача решается без нового кода, но маршрут и цена пляшут от прогона к прогону
- **routing**: триаж читает вопрос и сам передаёт его профильному агенту

Решаете вы

- Шаги защиты в код: предсказуемо и дёшево, но новый случай — новый код
- **deterministic**: первый пишет, второй проверяет и может оборвать цепочку
- **parallelization**: три прогона одного вопроса сразу, лучший выбирается
- **llm-as-a-judge**: второй агент возвращает первому замечания, пока не примет

Все четыре лежат готовым кодом в примерах SDK, папка `agent_patterns`

Сессия: продолжение разговора

БЕЗ СЕССИИ

```
Runner.run_sync(agent, "Сколько аптек в городе?")
Runner.run_sync(agent,
    "А сколько из них круглосуточных?")
```

Второй вопрос приходит к модели сам по себе: «из них» — из кого? Каждый вызов начинается с чистого листа

- Разговор переживает перезапуск: `SQLiteSession` кладёт историю в файл, а не в память процесса
- Разговоров может быть много: у каждого свой идентификатор, они не смешиваются
- Плата за удобство: история едет в **каждый** вызов и оплачивается заново, поэтому у сессии есть предел длины

С СЕССИЕЙ

```
session = SQLiteSession("client-17")

Runner.run_sync(agent, "Сколько аптек...",
    session=session)
Runner.run_sync(agent, "А сколько из них...",
    session=session)
```

Сессия сама подставляет прошлые сообщения в следующий вызов и дописывает туда новые

Контекст, которого модель не видит

ЧТО УХОДИТ МОДЕЛИ

История сообщений и схемы инструментов. За это платят токенами на каждом шаге

ЧТО ВИДИТ ТОЛЬКО ВАШ КОД

```
@dataclass
class Deps:
    db: Connection
    user_id: int
    shown: set[int] = field(default_factory=set)

@function_tool
def find_places(ctx: RunContextWrapper[Deps],
                kind: str) -> str:
    rows = ctx.context.db.execute(...)
    ctx.context.shown.update(r["id"] for r in rows)
    return render(rows)

Runner.run_sync(agent, вопрос,
                context=Deps(db=con, user_id=17))
```

- Модели этот объект **не уходит**: это зависимости для кода, так и написано в документации SDK
- Соединение с базой, ключи, идентификатор пользователя — всё, чему не место в промпте
- Его можно **менять по ходу**: один инструмент записал, следующий прочитал
- Там же лежит `usage` : инструмент видит, сколько потрачено, и может урезать выдачу

Ответ объектом, а не текстом

ЧТО ПРОСИМ

```
class Place(BaseModel):
    id: int
    name: str
    distance_m: int

class Answer(BaseModel):
    found: int
    places: list[Place]
    missing_in_data: list[str]

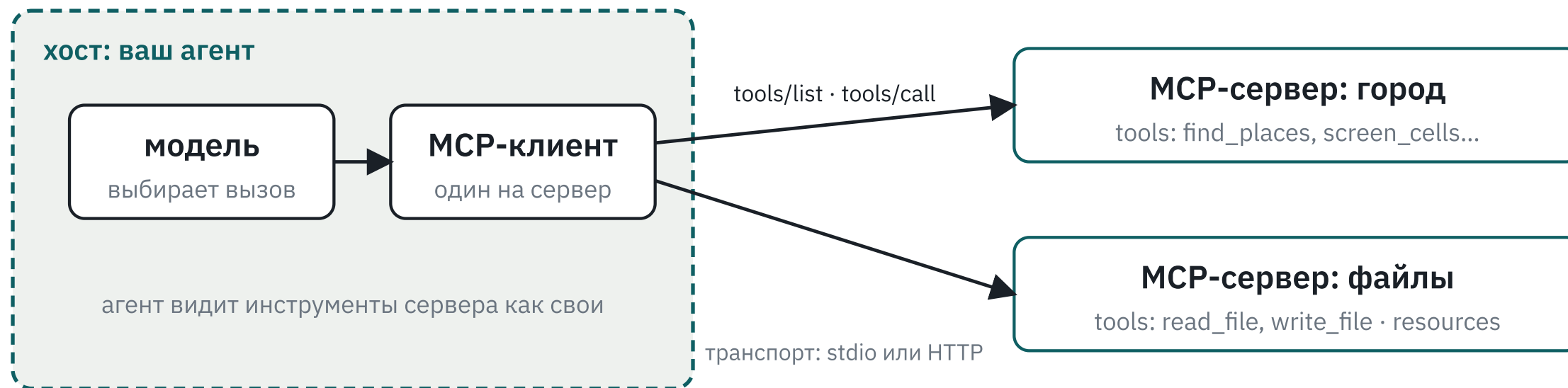
agent = Agent(..., output_type=Answer)
Runner.run_sync(agent,
    "три аптеки рядом с 47.2357, 39.7015")
```

ЧТО ПРИХОДИТ

```
Answer(
    found=3,
    places=[
        Place(id=663, name="-",
            distance_m=161),
        Place(id=1648, name="Вита Экспресс",
            distance_m=215),
        Place(id=685, name="Социальная аптека",
            distance_m=221)],
    missing_in_data=[
        "у первой аптеки нет названия",
        "часы работы известны у 40% мест"])
```

Это объект, а не текст: `answer.places[0].distance_m` равно `161`, и его можно сравнить с базой одной строкой

МСР: зачем общий стандарт



- Придумала **Anthropic**, анонс 25 ноября 2024, спецификация открытая
- Инструменты живут **отдельно от агента**: один сервер — любой агент, один агент — любые серверы
- Прimitives: **tools** (действия), **resources** (данные), **prompts** (шаблоны)
- Прижилось: в Agents SDK у агента есть параметр `mcp_servers`, хотя SDK не от Anthropic

Итог

- Вызов инструмента — дообученный формат текста; модель видит только имя, описание и схему
- Схема гарантирует валидность, но не смысл: смысл проверяет код и шаг проверки
- Описание и формат результата — часть интерфейса: их влияние видно в доле верных вызовов, а не в ощущениях
- Фреймворк забирает механику: цикл, историю, диспетчер вызовов, передачу между агентами
- И не забирает решения: описания, формат результата, что считать нарушением, что нельзя выбрасывать из истории
- Handoff отдаёт разговор, агент-как-инструмент оставляет его себе — выбор за задачей, а не за фреймворком

Дополнительное чтение

- Schick et al. «Toolformer: Language Models Can Teach Themselves to Use Tools» (2023) — arxiv.org/abs/2302.04761
- Wang et al. «Executable Code Actions Elicit Better LLM Agents» (CodeAct, ICML 2024) — arxiv.org/abs/2402.01030
- Berkeley Function Calling Leaderboard — gorilla.cs.berkeley.edu/leaderboard.html
- Anthropic «Writing effective tools for agents — with agents» (2025) — концептуальный текст про дизайн инструментов
- OpenAI «Function calling», Google «Function calling with the Gemini API» — документация
- Dong et al. «XGrammar» (2024) — как устроен constrained decoding — arxiv.org/abs/2411.15100
- Model Context Protocol — спецификация — modelcontextprotocol.io/specification
- TOON (Token-Oriented Object Notation) — формат выдачи для LLM: длина и поля объявляются один раз — github.com/toon-format/spec
- OpenAI Agents SDK — документация: агенты, инструменты, гардрейлы, handoffs, sessions — openai.github.io/openai-agents-python
- Готовые паттерны с кодом: routing, deterministic, parallelization, llm-as-a-judge — github.com/openai/openai-agents-python/tree/main/examples/agent_patterns